Laboratório de Concorrência · Aula 10

Daniel Morotti Vieira · RA 241914442 · Turma ESOFT6C
Gerado em 11/10/2026, 19:55:52 · tempo registrado no cronômetro: 34:22

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A diferença é que a sincronização de cooperação acontece quando uma tarefa precisa esperar outra terminar algo específico para conseguir avançar (tipo o atendente do Mcdonald's que precisa esperar o chapeiro fritar o hambúrguer pra poder montar o lanche), enquanto a sincronização de competição rola quando duas tarefas tentam usar o mesmo recurso compartilhado ao mesmo tempo e precisam de exclusão mútua (tipo duas pessoas tentando usar a mesma cabine de banheiro químico de uma festa, onde só entra uma por vez e tranca a porta pra outra não atropelar).

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

O esperado teoricamente seria 2.000.000 (pois são duas threads somando 1.000.000 cada). Porém, na prática, devido à condição de corrida nos três passos (ler, somar e gravar), o programa vai imprimir um valor aleatório e bem menor que 2.000.000.

1ª execução (n = 1.000.000)

1026504

2ª execução (n = 1.000.000)

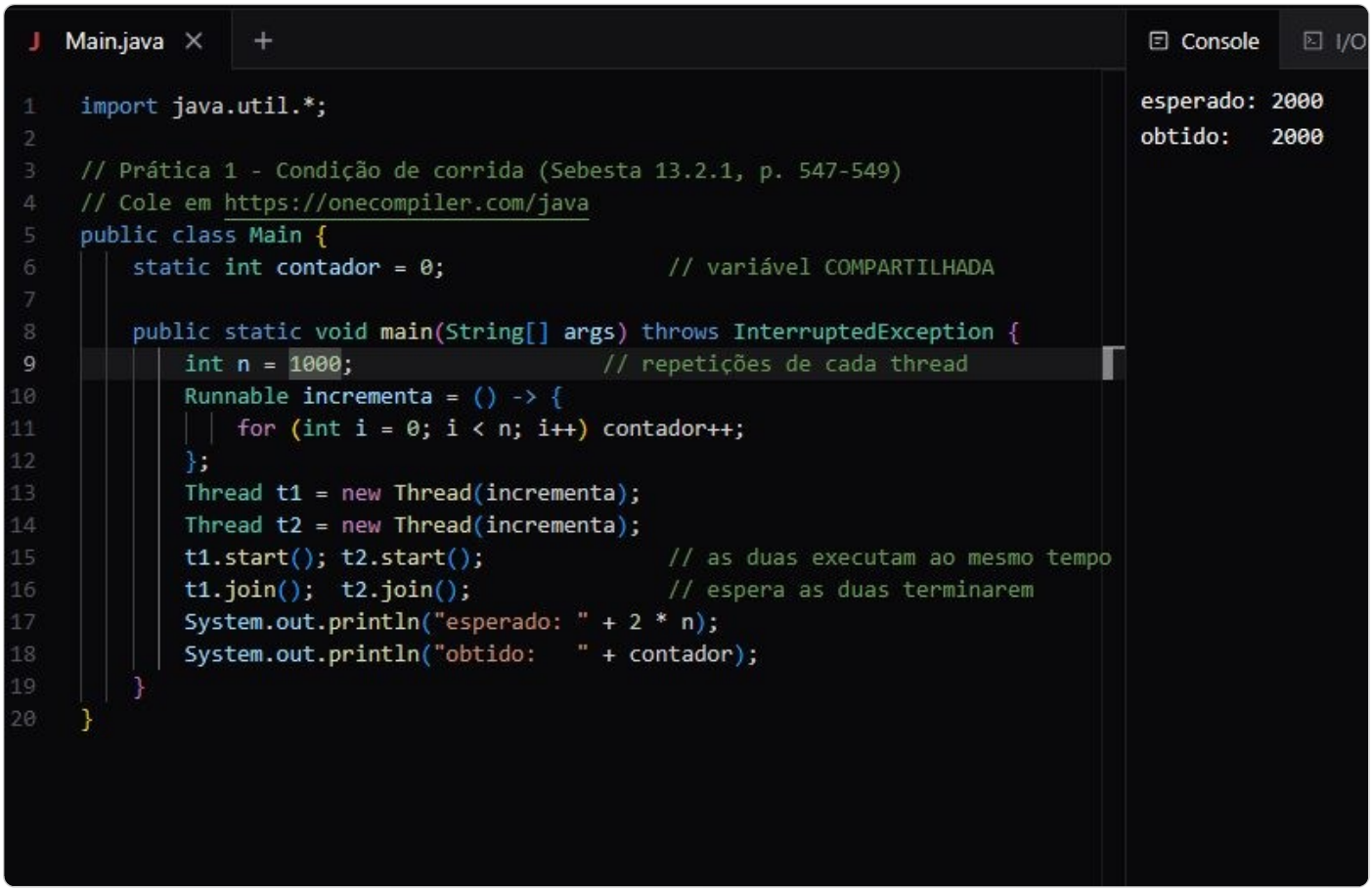
1007193

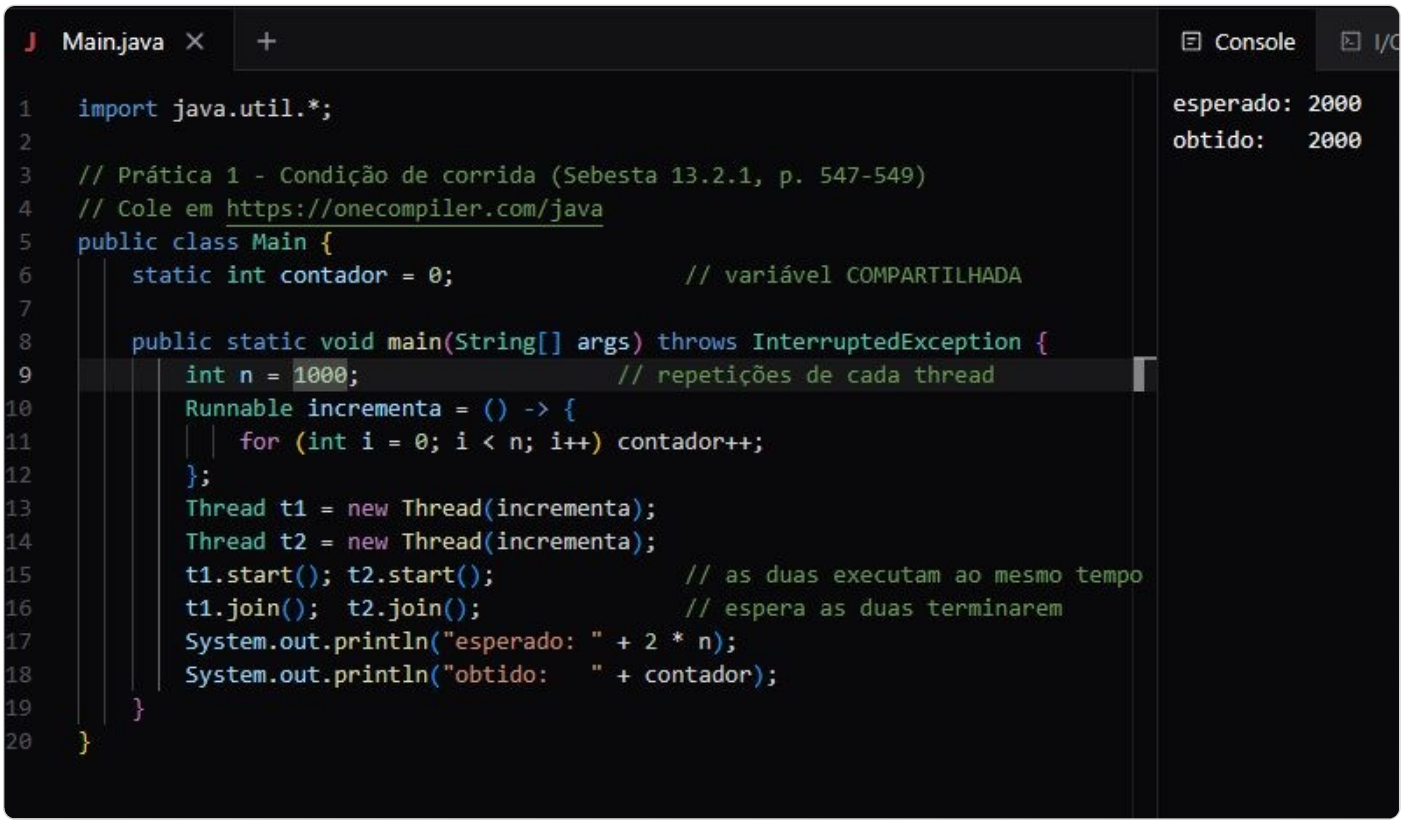
3ª execução (n = 1.000.000)

1968443

Saída do programa

não respondido





1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não, os valores não bateram com o esperado (que era 2.000.000) e mudaram a cada execução (1.026.504, 1.007.193 e 1.968.443). Isso acontece porque a troca de contexto entre as threads e a intercalação das instruções na CPU são imprevisíveis (não determinísticas). A cada rodada, as threads se "atropelam" em momentos

diferentes ao tentar atualizar a variável compartilhada.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

A operação contador++ é dividida em três passos: 1. Ler, 2. Somar e 3. Gravar.
A atualização se perde assim: O contador vale 10.A Thread 1 e a Thread 2 fazem o Passo 1 (Ler) ao mesmo tempo; ambas leem o valor 10.Ambas fazem o Passo 2 (Somar) na memória local e chegam ao resultado 11.A Thread 1 faz o Passo 3 (Gravar) e escreve 11 no contador.Logo em seguida, a Thread 2 faz o Passo 3 (Gravar) e escreve 11 no contador também.Como as duas leram 10 e gravaram 11, duas somas aconteceram, mas o contador só subiu 1. Uma das atualizações foi perdida.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000, a execução foi tão rápida que cada thread provavelmente terminou todo o seu loop de 1.000 repetições sem ser interrompida pelo processador, resultando em 2.000 cravado (dando a falsa ilusão de que está tudo certo). Isso faz desse tipo de bug algo muito perigoso: em testes locais ou com poucos dados, o código roda rápido, o erro não aparece e passa batido. Mas quando vai pra produção, com servidor pesado e carga alta, o bug estoura, gerando comportamentos imprevisíveis e difíceis de reproduzir e corrigir.

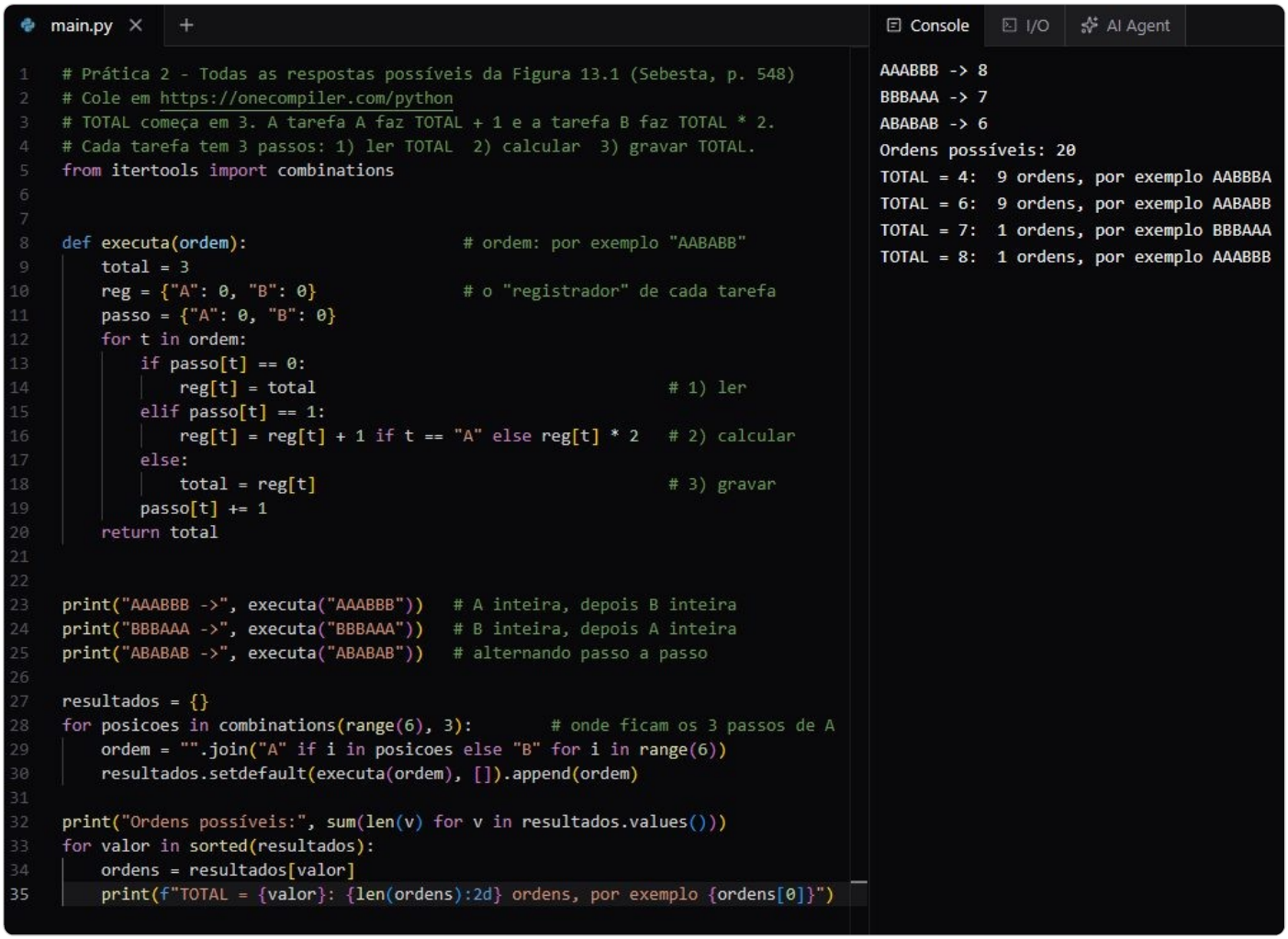
Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8
BBBAAA → 7
AABBBA → 4
ABABAB → 6

Saída do programa

não respondido



2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São possíveis 20 ordens no total e aparecem 4 resultados diferentes (4, 6, 7 e 8). Apenas 1 ordem dá o resultado "correto" (8), que é a ordem AAABBB (onde a tarefa A executa do início ao fim antes de B começar).

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Escolhendo a ordem AAABBA (ou AABBBA):

A lê TOTAL (3) e calcula 4 no seu registrador.

B lê TOTAL (3) e calcula 6 no seu registrador.

B grava 6 em TOTAL.

A grava o valor que tinha calculado lá no começo (4) em TOTAL.

O que aconteceu foi que a gravação final de A sobrescreveu completamente o resultado de B. A operação de B foi "anulada" e perdida porque A trabalhou com um valor antigo de TOTAL e gravou por último.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

A versão com AtomicInteger tende a ser a mais rápida entre as corrigidas por usar instruções atômicas direto no nível do processador (hardware/CAS) sem travar threads, enquanto a versão sem sincronização é a mais rápida de todas (porém incorreta por causa das perdas de atualização). Nenhuma das versões sincronicadas (synchronized e AtomicInteger) será incorreta; ambas garantem o resultado de 2.000.000.

Saída do programa

não respondido

Main.java

```java
// Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
// Cole em https://onecompiler.com/java
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static int semCuidado = 0, comSync = 0;
    static AtomicInteger atomico = new AtomicInteger();

    static synchronized void incSync() { comSync++; }   // uma thread por vez

    static long mede(Runnable corpo) throws InterruptedException {
        Thread a = new Thread(corpo), b = new Thread(corpo);
        long inicio = System.nanoTime();
        a.start(); b.start(); a.join(); b.join();
        return (System.nanoTime() - inicio) / 1_000_000;  // milissegundos
    }

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000;
        long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
        long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
        long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
        System.out.println("esperado:          " + 2 * n);
        System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
        System.out.println("synchronized:      " + comSync + " em " + ms2 + " ms");
        System.out.println("AtomicInteger:     " + atomico.get() + " em " + ms3 + " ms");
    }
}
```

Console I/O AI Agent

```
esperado:          2000000
sem sincronizacao: 1635782 em 46 ms
synchronized:      2000000 em 46 ms
AtomicInteger:     2000000 em 60 ms
```

Main.java

```java
// Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
// Cole em https://onecompiler.com/java
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static int semCuidado = 0, comSync = 0;
    static AtomicInteger atomico = new AtomicInteger();

    static synchronized void incSync() { comSync++; }   // uma thread por vez

    static long mede(Runnable corpo) throws InterruptedException {
        Thread a = new Thread(corpo), b = new Thread(corpo);
        long inicio = System.nanoTime();
        a.start(); b.start(); a.join(); b.join();
        return (System.nanoTime() - inicio) / 1_000_000;  // milissegundos
    }

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000;
        long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
        long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
        long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
        System.out.println("esperado:          " + 2 * n);
        System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
        System.out.println("synchronized:      " + comSync + " em " + ms2 + " ms");
        System.out.println("AtomicInteger:     " + atomico.get() + " em " + ms3 + " ms");
    }
}
```

Console I/O AI Agent

```
esperado:          2000000
sem sincronizacao: 1439506 em 2 ms
synchronized:      2000000 em 97 ms
AtomicInteger:     2000000 em 70 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões synchronized e AtomicInteger deram o valor correto de 2.000.000. As versões corretas costumam ser mais lentas que a versão sem sincronização porque exigem overhead de sincronização (como controle de travas, alternância/bloqueio de threads e operações atômicas no barramento de memória), o que impede as threads de executarem totalmente livres e em paralelo puro.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve. Ter um AtomicInteger para a conta A e outro para a conta B garante apenas que cada saldo individual seja atualizado de forma atômica, mas não garante a atomicidade da operação completa de transferência. Uma thread poderia tirar dinheiro da conta A e, antes de conseguir depositar na conta B, outra thread ler os dois saldos e enxergar um estado inconsistente (dinheiro "sumido" temporariamente). Para resolver isso, seria necessário garantir exclusão mútua na operação inteira usando um bloco sincronizado (synchronized) ou travas (Lock/ReentrantLock) cobrindo as duas contas ao mesmo tempo, aplicando uma ordem fixa de bloqueio para evitar impasses (deadlocks).

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

não respondido

Saídas (antes e depois da correção)

não respondido

Main.java

```java
// Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
// Cole em https://onecompiler.com/java
import java.lang.management.ManagementFactory;

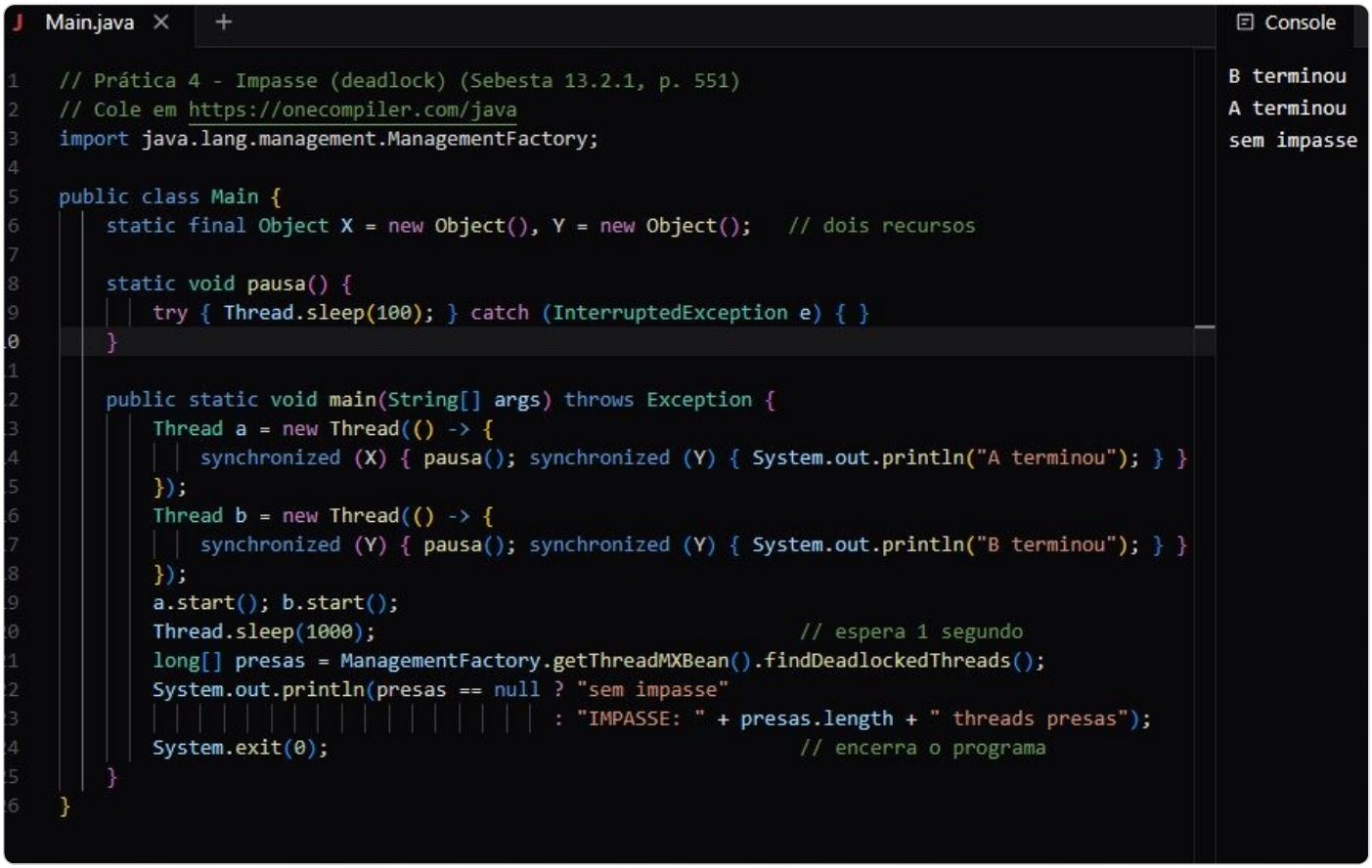
public class Main {
    static final Object X = new Object(), Y = new Object();   // dois recursos

    static void pausa() {
        try { Thread.sleep(100); } catch (InterruptedException e) { }
    }

    public static void main(String[] args) throws Exception {
        Thread a = new Thread(() -> {
            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
        });
        Thread b = new Thread(() -> {
            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
        });
        a.start(); b.start();
        Thread.sleep(1000);                                  // espera 1 segundo
        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
        System.out.println(presas == null ? "sem impasse"
                                          : "IMPASSE: " + presas.length + " threads presas");
        System.exit(0);                                      // encerra o programa
    }
}
```

Console I/O AI Age

```
IMPASSE: 2 threads presas
```



4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas por causa de uma espera circular pelos recursos. A Thread A segurava a trava do recurso X e ficou esperando a liberação do recurso Y. Ao mesmo tempo, a Thread B segurava a trava do recurso Y e ficou esperando a liberação do recurso X. Como nenhuma das duas solta o recurso que já tem antes de conseguir o outro, as duas ficam travadas para sempre.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem elimina a possibilidade de espera circular (uma das condições necessárias para o deadlock). Se ambas as threads tentam adquirir X primeiro, a primeira thread que conseguir travar X vai poder rodar, pegar Y e finalizar, enquanto a outra thread fica esperando na fila de X sem prender nenhum outro recurso. Exemplo de sistema real: Um sistema de transferência bancária entre contas. Se a conta A tenta transferir para B (travando A e depois B) ao mesmo tempo que B tenta transferir para A (travando B e depois A), ocorre um impasse. Definindo uma ordem fixa (por exemplo, sempre travar primeiro a conta que possui o menor ID/número de conta), o impasse é completamente evitado.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores: Espera-se cerca de 200 ms (6 tarefas de 100 ms divididas igualmente por 3 trabalhadores = 2 tarefas por trabalhador). Com 1 trabalhador: Espera-se cerca de 600 ms (1 trabalhador faz as 6 tarefas em sequência: $6 \times 100\text{ ms}$). Com 6 trabalhadores: Espera-se cerca de 100 ms (cada um dos 6 trabalhadores pega 1 tarefa ao mesmo tempo).

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

não respondido

main.go

```go
// Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
// Cole em https://onecompiler.com/go
package main

import (
    "fmt"
    "sync"
    "time"
)

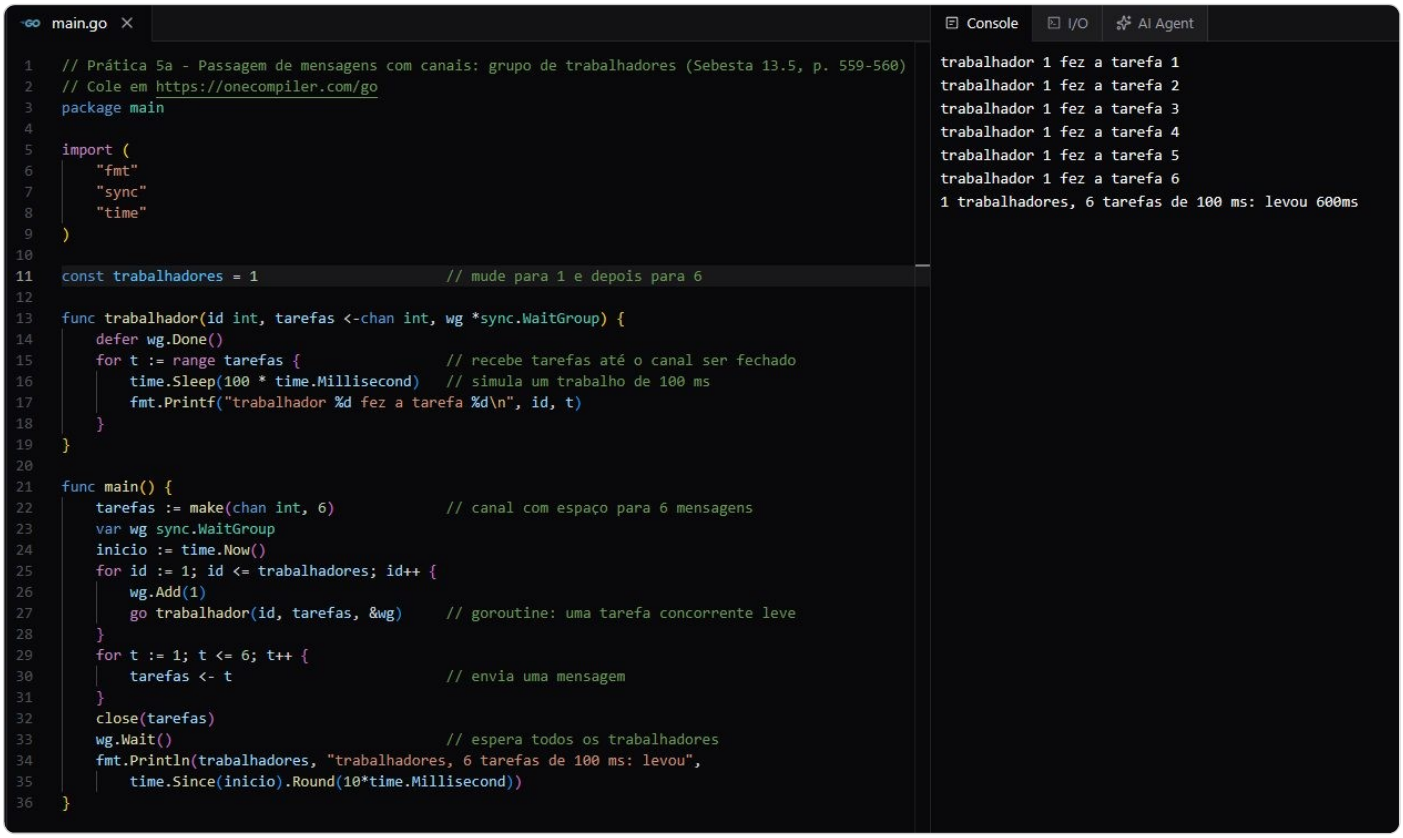
const trabalhadores = 3                     // mude para 1 e depois para 6

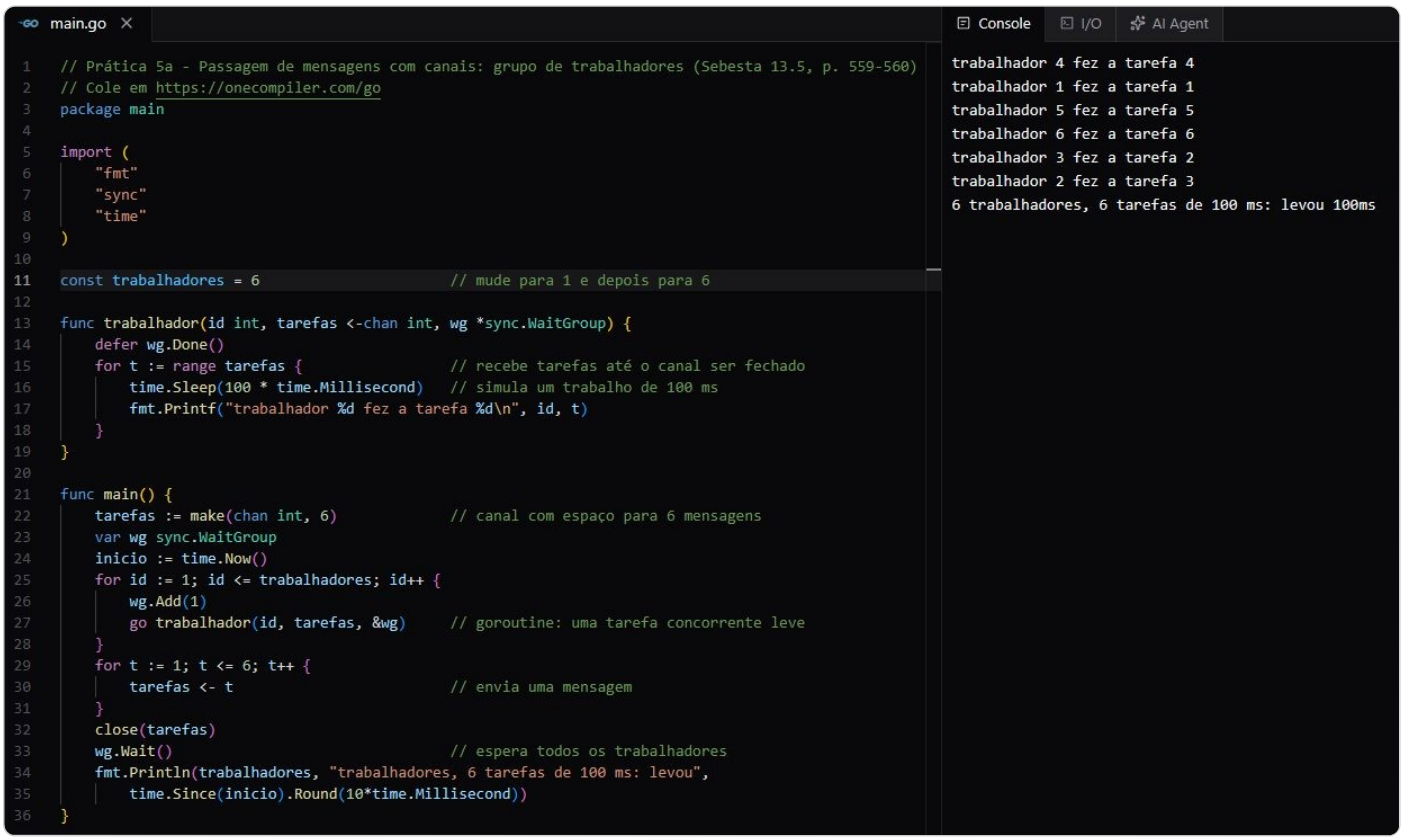
func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
    defer wg.Done()
    for t := range tarefas {                // recebe tarefas até o canal ser fechado
        time.Sleep(100 * time.Millisecond)  // simula um trabalho de 100 ms
        fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
    }
}

func main() {
    tarefas := make(chan int, 6)            // canal com espaço para 6 mensagens
    var wg sync.WaitGroup
    inicio := time.Now()
    for id := 1; id <= trabalhadores; id++ {
        wg.Add(1)
        go trabalhador(id, tarefas, &wg)    // goroutine: uma tarefa concorrente leve
    }
    for t := 1; t <= 6; t++ {
        tarefas <- t                        // envia uma mensagem
    }
    close(tarefas)
    wg.Wait()                               // espera todos os trabalhadores
    fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
        time.Since(inicio).Round(10*time.Millisecond))
}
```

Console | I/O | AI Agent

```
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 2 fez a tarefa 6
trabalhador 3 fez a tarefa 4
trabalhador 1 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```



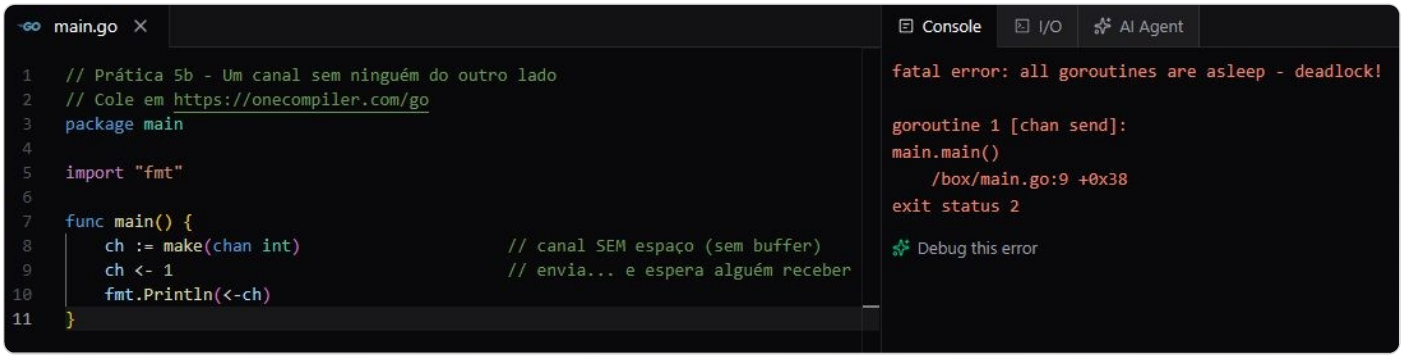


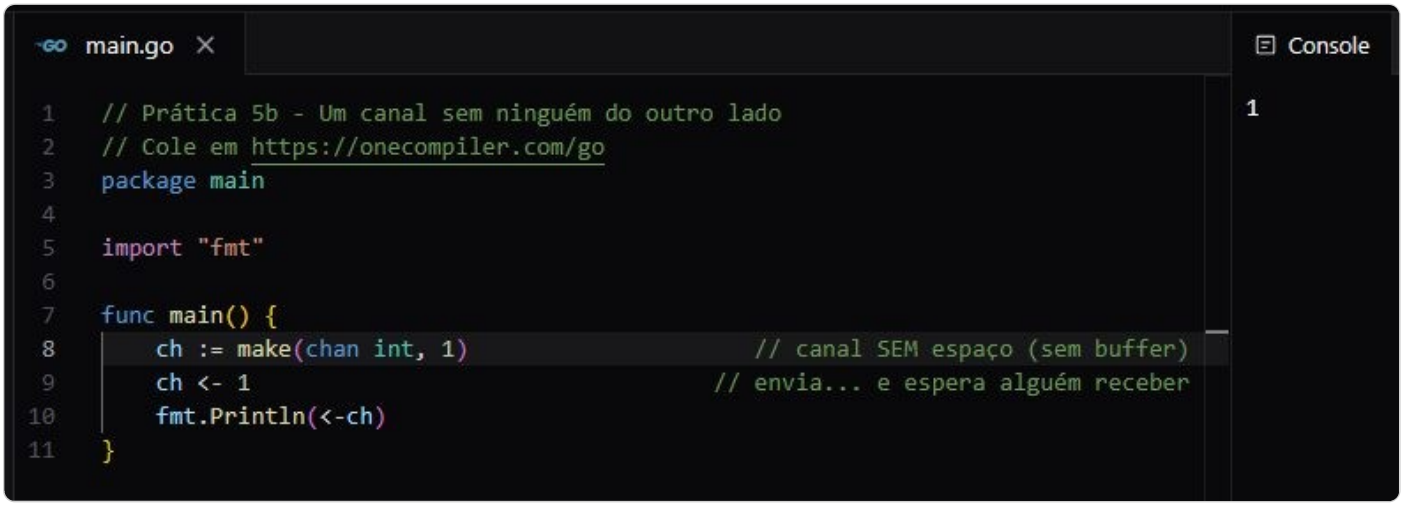
5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Os tempos mostram o impacto do paralelismo: com 1 trabalhador leva 600 ms, com 3 cai para 200 ms e com 6 cai para 100 ms. Este programa não precisa de synchronized nem de travas porque não há memória compartilhada sendo alterada ao mesmo tempo. As goroutines comunicam-se enviando cópias de dados através do canal, e o próprio canal de Go encarrega-se internamente de sincronizar quem envia e quem recebe de forma segura.

Saídas (antes e depois da troca)

não respondido





5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque um canal sem buffer (make(chan int)) exige um rendezvous (sincronização síncrona): quem envia (ch <- 1) fica bloqueado esperando até que outra goroutine esteja pronta para receber. Como a leitura (<-ch) estava na mesma thread/goroutine e vinha logo abaixo na linha seguinte, o programa travava na linha do envio e nunca chegava na leitura, gerando deadlock. O canal com espaço para 1 mensagem (make(chan int, 1)) resolve porque o envio guarda o valor no buffer imediatamente sem precisar de esperar por um recetor, permitindo que a linha de envio conclua e o programa avance até à linha de leitura sem bloquear.

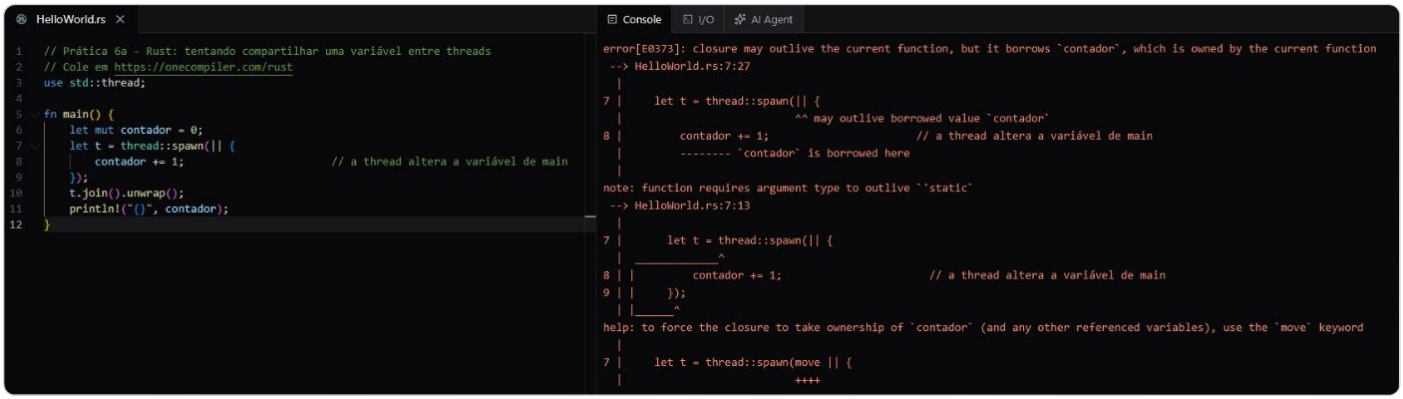
Prática 6: O compilador como guarda

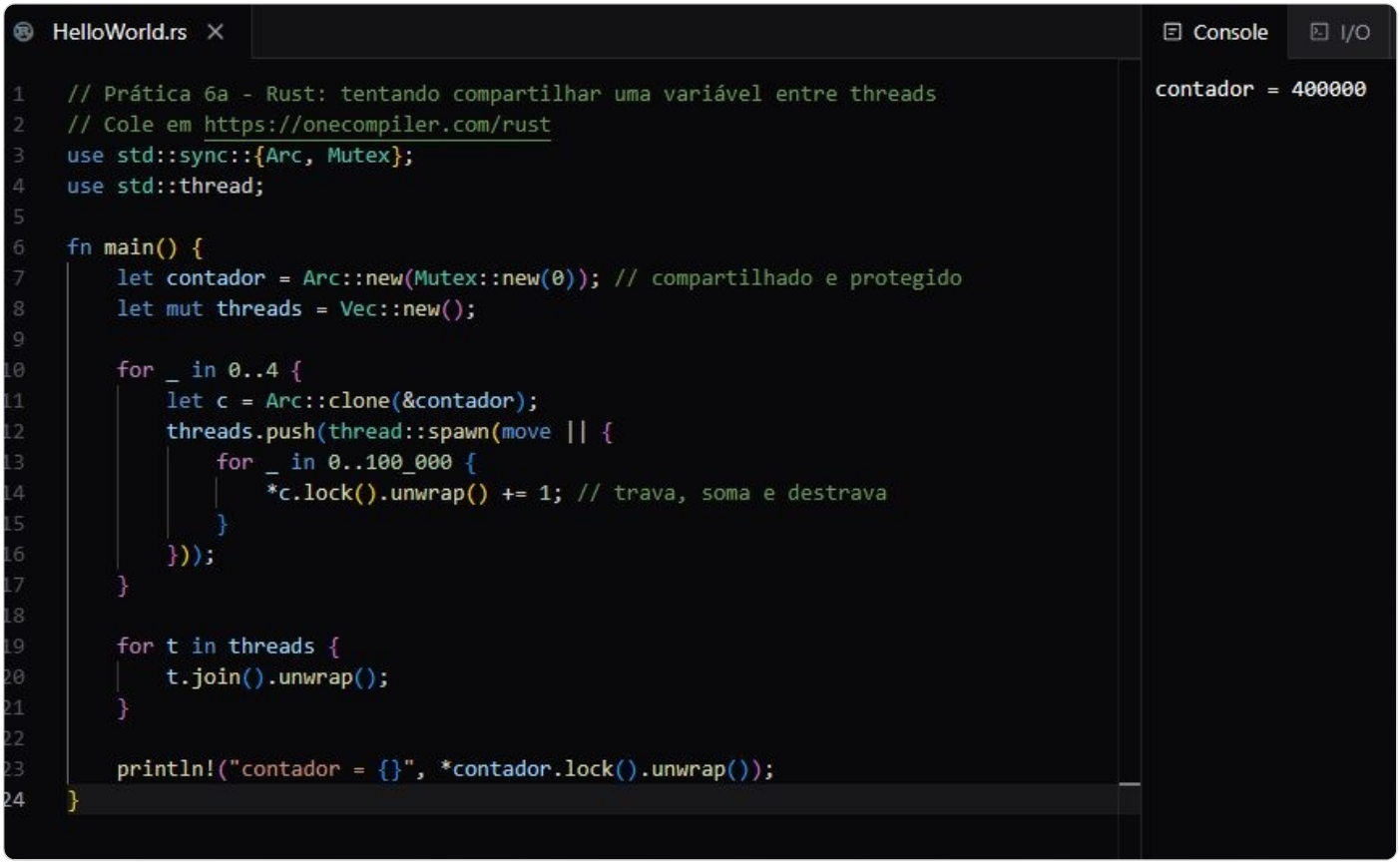
Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

O programa p6a_rust_erro.rs não vai compilar. O compilador de Rust vai recusar o código com um erro de borrow checker, pois a thread tenta capturar uma referência mutável de contador da função main que pode não viver o tempo suficiente (ou sem garantia de segurança entre threads).

Saídas (o erro e o resultado com move)

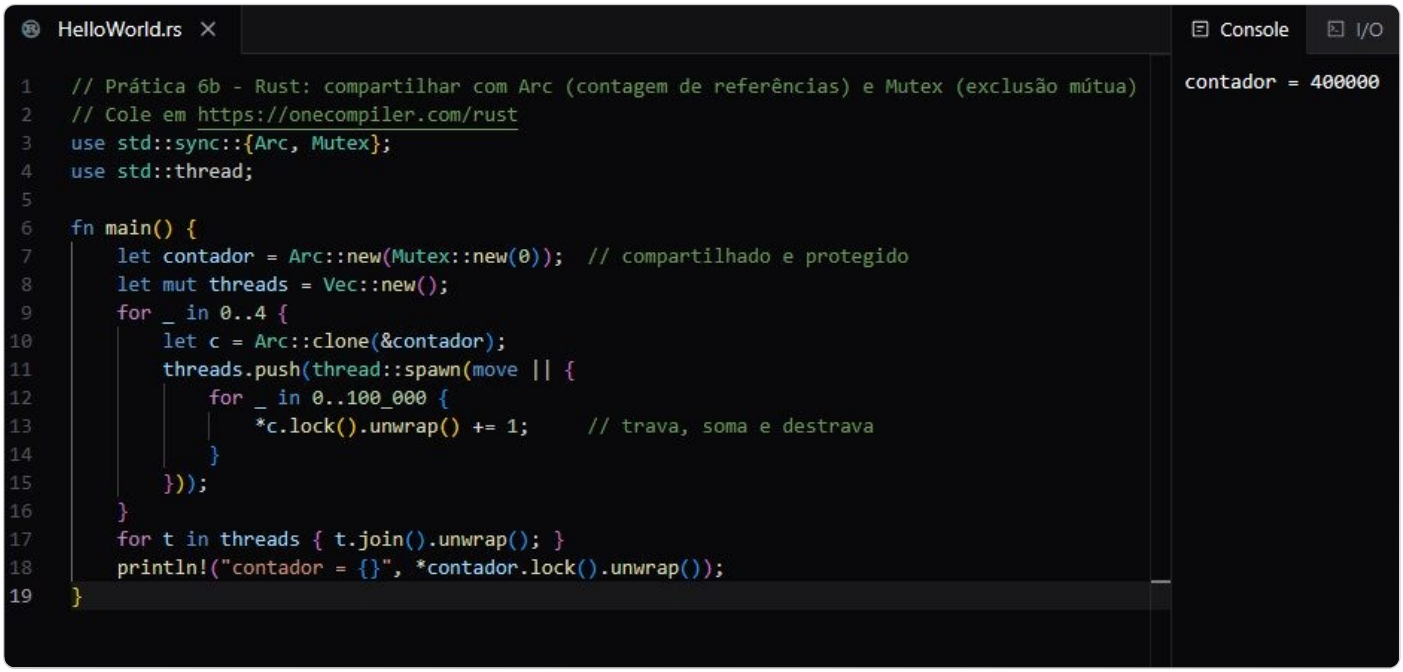
não respondido





Saída do programa

não respondido



6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Como o tipo primitivo i32 implementa a trait Copy, a palavra-chave move faz com que a closure da nova thread receba uma cópia independente da variável contador em vez de alterar a variável original da main. A thread incrementa a sua própria cópia local (que vai para 1 e é descartada no fim da thread), enquanto o contador da main permanece intacto com o valor 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro de corrida de dados ocorreu em runtime (durante a execução), gerando valores incorretos e silenciosos que variavam a cada rodada. Em Rust, o erro foi detetado em compile-time (durante a compilação), onde o compilador recusou-se a gerar o executável sem as devidas proteções (Arc e Mutex). Vantagem: Elimina completamente bugs de condição de corrida em produção, garantindo segurança de memória e concorrência antes mesmo de rodar o código. Custo: Curva de aprendizagem mais acentuada e maior rigidez na escrita do código, exigindo lidar explicitamente com regras de posse (ownership), referências (Arc) e travas (Mutex).

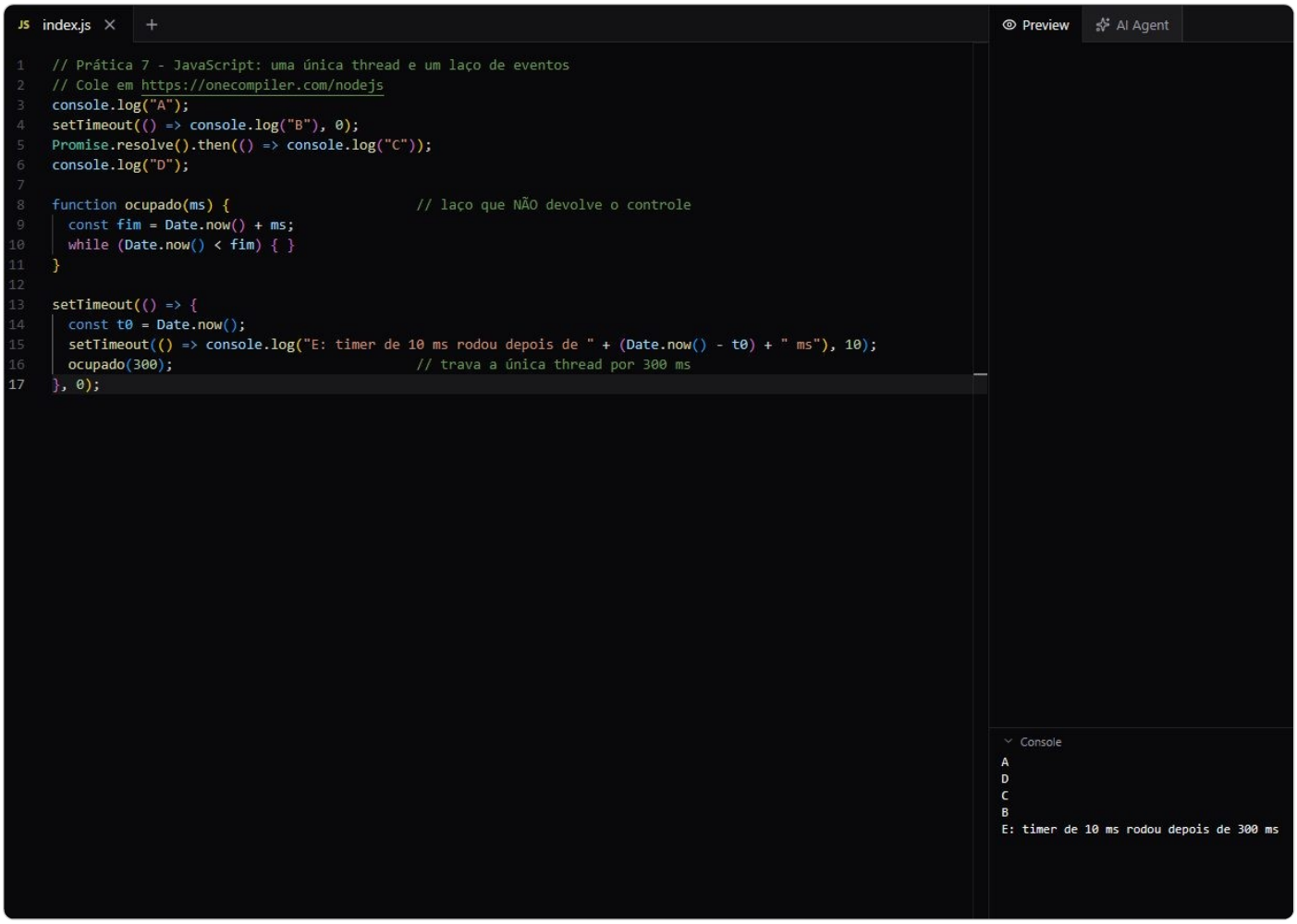
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A ordem impressa para as letras será A, D, C e B.

Saída do programa

não respondido



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

As letras saíram na ordem A, D, C, B. D saiu antes de B e C porque faz parte do código síncrono principal (call stack), que tem prioridade total e executa até o fim primeiro. C (Promise) saiu antes de B (setTimeout) porque as promessas entram na fila de microtarefas (microtasks), que são processadas imediatamente após o código síncrono e antes de qualquer macrotarefa. B saiu por último porque o setTimeout entra na fila de macrotarefas (macrotasks/task queue), que só é executada depois que todas as microtarefas forem concluídas.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms rodou somente após cerca de 300 ms porque a função ocupado(300) executou um loop síncrono travando a única thread do JavaScript por 300 ms. Como o Event Loop só consegue pegar a próxima tarefa da fila quando a thread fica totalmente livre, o timer teve de esperar o término desse bloco síncrono. Para um servidor Node.js que atende múltiplos utilizadores, isso significa um gargalo grave: um cálculo pesado síncrono congela a thread inteira, fazendo com que todos os outros utilizadores fiquem bloqueados e esperando até que esse cálculo termine.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer uma condição de corrida (race condition), levando ao problema de overbooking (venda duplicada do mesmo bilhete), onde o sistema lê que há 1 lugar disponível para ambas as requisições antes de atualizar o saldo e confirma a compra para as duas pessoas ao mesmo tempo. Evitaria utilizando sincronização de competição com exclusão mútua no backend (como um bloco de código sincronizado, locks ou transações atómicas com pessimistic/optimistic locking na base de dados) para garantir que apenas uma transação de compra consulte e decremente o lugar de cada vez.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi ver como o compilador de Rust consegue barrar condições de corrida antes mesmo da execução, enquanto em Java o mesmo erro passa despercebido e gera resultados completamente imprevisíveis.